# 설비 고장을 예측하는 AI 데이터 분석

## NumPy 기초

데이터 추출과 통계 분석

posco

CONTENTS

# 목차


01 인덱싱과 슬라이싱 — 값 추출과 행·열 선택

02 배열 연산과 필터링 — 벡터 연산, 조건 필터링

03 기초 통계 — 평균·표준편차, axis 방향


posco

posco

# 분석 대상 공정 — CNC 가공(MCT)

**공구를 회전시켜 금속을 깎는 정밀가공 설비 — 회전수·토크·온도·전류를 함께 기록**

01 · 공정

## 스핀들을 돌려 금속을 깎는 설비

한 번의 가공마다
회전수·토크·온도·전류·사이클타임이 한 줄로 남음

02 · 핵심 값

## 회전수와 토크 (속도와 깎는 힘)

정상 가공은 고회전 유지
회전수가 크게 떨어진 시점이 이상 신호

03 · 이 데이터

## 40행 6열
## 회전수 평균 4212.6

회전수가 2000 아래로 떨어진 시점 4건
이 4건을 찾아내는 것이 목표

출처: KAMP 제조AI 데이터셋 — MCT 가공 Tool 관리 (중소기업기술정보진흥원·KAIST). 교육용 발췌.

posco

# 01

# 인덱싱과 슬라이싱

값 추출과 행·열 선택

# 1차원 인덱싱

**배열을 만들었으면 그 안에서 원하는 값을 꺼낼 줄 알아야 함**

01 • 값 추출의 시작

## 배열에서 원하는 값 하나를 꺼내는 가장 기본 방법

앞서 배열을 만들고 모양을 다뤘다면, 이제부터는 그 안에서 원하는 값만 콕 집어 꺼내기.

02 • 인덱싱의 정의

## 대괄호와 번호로 특정 위치의 값 하나를 꺼내기

번호는 인덱스라고 부름. 다음 슬라이드에서 실제 코드로 확인.

# 1차원 인덱싱 — 번호로 값 꺼내기




-1은 개수를 몰라도 항상 맨 마지막 값 — 최신 값 꺼내기에 편함

posco

# 1차원 인덱싱 — 번호로 값 꺼내기

**번호는 0부터 시작, 음수는 뒤에서부터**

CODE

```
temp = np.array([70, 72, 71, 95, 73])
print(temp[0])
# 출력: 70 (첫 번째)
print(temp[-1])
# 출력: 73 (맨 마지막)
```

-1은 항상 맨 마지막 값 · 데이터 개수와 상관없이 최신 값 꺼내기에 편함

# 1차원 슬라이싱




1:4는 1·2·3번(끝 4 제외) · ::2는 하나 걸러 하나씩 · 시작·끝을 생략하면 처음부터 또는 끝까지

posco

# 1차원 슬라이싱

**시작:끝으로 구간 잘라내기 (끝 번호 제외)**

CODE

```
temp = np.array([70, 72, 71, 95, 73, 68])
print(temp[1:4])
# 출력: [72 71 95]
print(temp[::2])
# 출력: [70 71 73]
```

1:4는 1·2·3번(끝 4 제외) · ::2는 하나 걸러 하나씩 · 시작·끝을 생략하면 처음부터 또는 끝까지

# 2차원 인덱싱

**행과 열을 함께 지정 — 앞=행, 뒤=열**

CODE

```
data = np.array([[70, 2.1], [72, 2.3]])
print(data[0, 1])
# 출력: 2.1 (0행 1열)
```

쉼표로 행·열을 한 번에 지정 —항상 행이 먼저, 열이 나중. shape과 같은 순서이며 번호는 0부터

posco

# 2차원 슬라이싱 — 행•열 선택

**행 전체, 열 전체, 일부 구간 잘라내기 — 콜론이 전부를 의미**

CODE

```
data = np.array([[70, 2.1], [72, 2.3]])
print(data[0])
# 출력: 행 전체
print(data[:, 0])
# 출력: 0열 전체
```

data[0] → 0행 전체

| | |
|---|---|
| **4603** | **2** |
| 58 | 5 |
| 1216 | 1 |

행은 배열의 기본 단위 — 번호만으로 꺼냄

data[:, 0] → 0열 전체

| | |
|---|---|
| **4603** | 2 |
| **58** | 5 |
| **1216** | 1 |

모든 행을 가로질러 모아야 해서 콜론이 필요

행은 배열의 기본 단위라 번호만으로 추출 —열은 모든 행을 가로질러 모아야 해서 콜론이 필요

# 인덱싱·슬라이싱 정리

**목적에 따라 표현 선택 — 번호 0부터, 끝은 제외 (1·2차원 공통)**

| 목적 | 표현 | 결과 |
|---|---|---|
| **값 하나** | `번호 1개 · a[2]` | 2번 값 |
| **구간** | `시작:끝 · a[1:4]` | 1~3번 |
| **행 전체** | `행 번호 · data[0]` | 0행 모두 |
| **열 전체** | `콜론, 열번호 · data[:, 0]` | 0열 모두 |

두 규칙은 1차원이든 2차원이든 동일 — 0부터 시작, 끝은 제외

posco

# 자주 하는 세 가지 실수

**세 가지만 조심하면 추출에서 겪는 어려움 대부분이 사라짐**

MISTAKE 01

## 번호를 1부터 세기

항상 **0부터** 시작. 첫 번째 값은 1번이 아니라 0번.

MISTAKE 02

## 슬라이싱 끝 번호 포함 착각

끝은 **제외**. 1:4는 4번을 포함하지 않음.

MISTAKE 03

## 2차원에서 행·열 순서 바꿈

항상 **행이 먼저**. shape과 같은 순서로 일관 유지.

# 실습 1. 특정 구간 꺼내기

**목표**

회전수 묶음에서 특정 시점 값과 일정 간격의 값을 꺼내기

**단계**

① 연속으로 측정한 회전수 열 개를 한 묶음으로 준비

② 맨 앞과 맨 뒤 값을 각각 꺼내기 (뒤쪽은 개수를 세지 않고)

③ 두 번째부터 네 번째까지 이어진 구간을 잘라내기

④ 하나 걸러 하나씩 값을 골라내기

**예상 결과**

맨 앞 4603, 맨 뒤 4603, 두 번째부터 네 번째까지 이어진 구간, 건너뛴 다섯 개가 각각 출력

posco

# 실습 1. 저회전 구간 찾아 꺼내기 정답

**뒤에서 세는 번호를 쓰면 데이터 개수를 몰라도 마지막 값을 꺼낼 수 있다**

CODE

```
import numpy as np

# 19~28번째 측정 회전수 (실측값)
rpm = np.array([4603, 4603, 58, 4603, 4603,
                1216, 4603, 4603, 1217, 4603])
print(rpm[0], rpm[-1])   # 4603 4603
print(rpm[1:4])          # [4603   58 4603]
print(rpm[::2])          # [4603   58 4603 4603 1217]
```

posco

# 실습 2. 한 시점과 한 센서 따로 꺼내기

**목표**

시점별 회전수•토크 표에서 한 시점과 한 센서를 각각 꺼내기

**단계**

① 네 시점의 회전수와 토크를 시점마다 한 줄씩 묶어 표로 준비

② 세 번째 시점의 값을 통째로 꺼내기

③ 모든 시점의 회전수만 세로로 모아 꺼내기

④ 모든 시점의 토크만 세로로 모아 꺼내기

⑤ 세 결과를 출력해 행과 열을 꺼내는 방식이 왜 다른지 확인

**예상 결과**

세 번째 시점 한 줄, 회전수 열 네 개, 토크 열 네 개가 각각 출력

posco

# 실습 2. 한 시점과 한 센서 따로 꺼내기 정답

**행은 번호만으로, 열은 모든 행을 가로질러야 해서 콜론이 필요하다**

CODE

```
import numpy as np

# 4시점 × 2센서(회전수, 토크) — 실측값
data = np.array([[4603, 2], [58, 5],
                 [1216, 1], [4967, 21]])
print(data[2])      # [1216    1]
print(data[:, 0])   # [4603   58 1216 4967]
print(data[:, 1])   # [ 2  5  1 21]
```

posco

# 02

# 배열 연산과 필터링

벡터 연산 · 조건 필터링

posco

# 배열 산술 연산

**두 배열을 같은 위치끼리 한 번에 계산**

CODE

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a + b)
# 출력: [11 22 33]
```

같은 위치 값끼리 계산 —두 배열의 크기가 다르면 위치를 맞출 수 없어 오류 발생

posco

# 스칼라 연산

**배열 전체에 숫자 하나를 한꺼번에 적용 — 섭씨→화씨 한 줄**

CODE

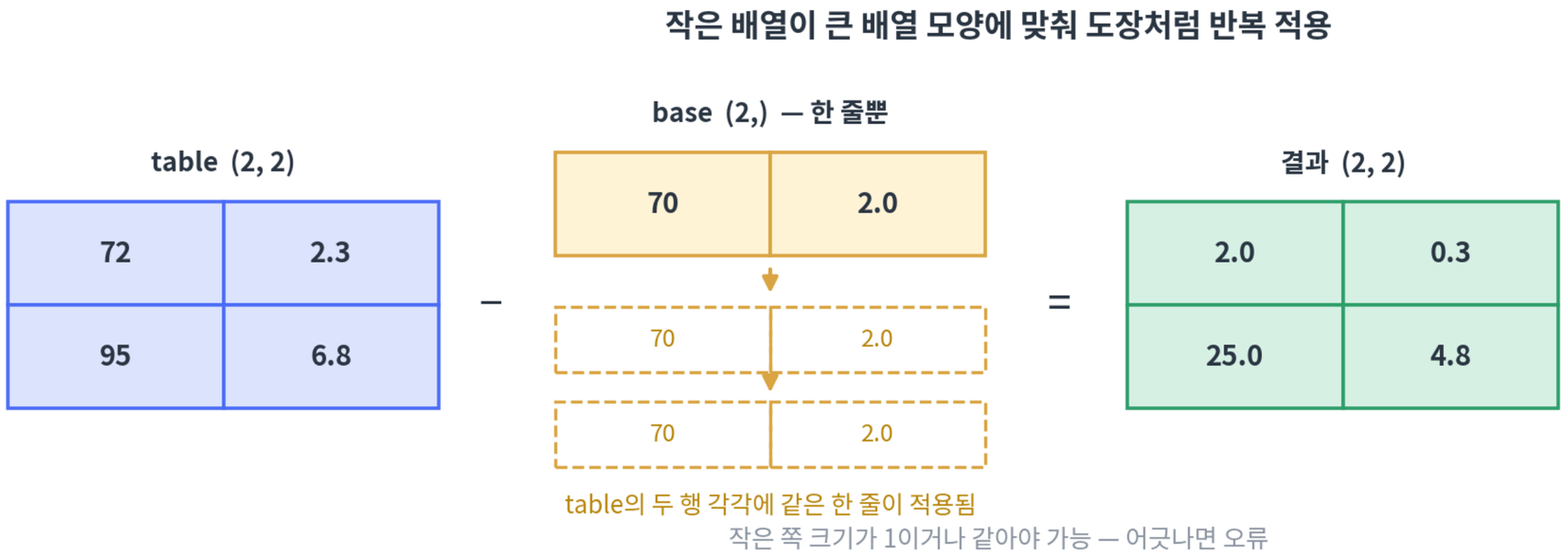
```
celsius = np.array([20.0, 25.0, 30.0])
print(celsius * 1.8 + 32)
# 출력: [68. 77. 86.]
```

크기를 맞출 필요 없이 모든 값에 적용 —연산 순서는 수학과 같아 곱하기가 먼저

posco

# 브로드캐스팅

**한 줄짜리 기준값이 모든 행에 퍼져서 계산**



posco

# 브로드캐스팅

**한 줄짜리 기준값이 모든 행에 퍼져서 계산**

CODE

```
table = np.array([[72, 2.3], [95, 6.8]])
base = np.array([70, 2.0])
print(table - base)
# 출력: 각 행에서 기준값 빼기
```

각 센서가 기준에서 얼마나 벗어났는지 모든 시점에 대해 한 번에 계산

posco

# 브로드캐스팅의 원리

**작은 것이 큰 것 모양에 맞춰 도장처럼 반복 적용**

01 · 숫자 하나

### 모든 위치로 퍼짐 (스칼라 연산)

하나의 숫자가 배열의 모든 자리에 똑같이 적용

02 · 한 줄 배열

### 모든 행으로 퍼짐 (기준값 빼기)

한 줄짜리 배열이 표의 모든 행에 동일하게 적용

03 · 조건

### 작은 쪽 크기가 1이거나 같아야

아무 때나 가능하지 않음. 크기가 어긋나면 오류 발생

도장을 여러 번 찍듯 작은 배열이 큰 배열 모양에 맞춰 반복

# 벡터 연산 정리

**반복문 없이 배열 전체를 한 번에 계산 — NumPy가 빠른 이유**

| 종류 | 대상 | 예시 |
| --- | --- | --- |
| **요소별** | 배열 + 배열 | `a + b` |
| **스칼라** | 배열 + 숫자 | `a * 2` |
| **브로드캐스팅** | 큰 + 작은 | `table - base` |

앞 교안에서 "묶음 계산"이라 부르던 것이 바로 이 벡터 연산 — 개별 값이 아니라 배열 전체를 대상으로 생각하는 것이 요령

# 실습 3. 회전수를 0과 1 사이로 맞추기

**목표**

단위가 큰 회전수를 0과 1 사이 값으로 정규화

**단계**

① 회전수 열 개 묶음을 준비

② 묶음에서 가장 작은 값과 가장 큰 값을 각각 구하기

③ 모든 값에서 가장 작은 값을 빼기

④ 그 결과를 최댓값과 최솟값의 차이로 나누기

⑤ 소수점 둘째 자리까지 다듬어 출력하고 0과 1이 하나씩 있는지 확인

**예상 결과**

가장 작은 값이 0, 가장 큰 값이 1이 되는 열 개짜리 배열 출력

posco

# 실습 3. 회전수를 0과 1 사이로 맞추기 정답

숫자 하나를 배열 전체에 적용하는 계산이 두 번 겹쳐 일어남 — 최솟값·최댓값 함수는 PART 03에서 자세히

CODE

```
import numpy as np

# 19~28번째 측정 회전수 (실측값)
rpm = np.array([4603, 4603, 58, 4603, 4603,
                1216, 4603, 4603, 1217, 4603])
norm = (rpm - rpm.min()) / (rpm.max() - rpm.min())
print(np.round(norm, 2))
# [1.   1.   0.   1.   1.   0.25 1.   1.   0.26 1.  ]
```

# 비교 연산과 불리언 배열

**조건을 걸면 참거짓 배열이 만들어짐 — 같다 비교는 등호 두 개**

CODE

```
vals = np.array([70, 95, 71, 88, 73])
print(vals > 85)
# 출력: [False True False True False]
```

원래 배열과 같은 크기의 참거짓 배열 — 참은 1, 거짓은 0으로 계산되므로 더하면 개수, 평균을 내면 비율

posco

# Boolean indexing

**불리언 배열로 조건에 맞는 값만 골라내기**

CODE

```
vals = np.array([70, 95, 71, 88, 73])
print(vals[vals > 85])
# 출력: [95 88]
```




조건이 참거짓 배열을 만들고 그것으로 값을 골라내는 두 단계 — 참인 자리만 남고 거짓은 버려짐

posco

# np.where 조건 처리

조건에 따라 값을 둘 중 하나로 바꾸기 — 조건·참·거짓 세 가지 인자

CODE

```
vals = np.array([70, 95, 71, 88, 73])
print(np.where(vals > 85, 1, 0))
# 출력: [0 1 0 1 0]
```

모든 값을 유지하면서 위험·정상 꼬리표 붙이기 — 위험은 1, 정상은 0

# 다중 조건 결합

**각 조건을 괄호로 묶고 & (그리고) 또는 | (또는) 로 연결**

CODE

```
vals = np.array([70, 95, 71, 88, 73])
print(vals[(vals > 70) & (vals < 90)])   # 둘 다 만족
# 출력: [71 88 73]
print(vals[(vals < 71) | (vals > 90)])   # 하나만 만족해도
# 출력: [70 95]
```

각 조건은 반드시 괄호로 — 기호가 비교보다 먼저 계산되려 해 괄호가 없으면 순서가 꼬임

# 조건 필터링 정리

**비교 •골라내기 •바꾸기 •묶기 —네 가지로 거의 모든 조건 처리 가능**

| 방법 | 하는 일 | 결과 크기 |
| --- | --- | --- |
| **비교 연산** | 참거짓 판단 | 그대로 |
| **불리언 인덱싱** | 값 골라내기 | 줄어듦 |
| **np.where** | 값 바꾸기 | 그대로 |
| **다중 조건** | 조건 묶기 | 경우별 |

고를 것인가 표시할 것인가 —불리언 인덱싱은 결과가 줄고, np.where는 크기를 그대로 유지

# 조건 필터링과 안전관제

**"경험상 위험하다"를 "데이터상 몇 %가 기준을 넘었다"로 — 진짜 분석의 시작**

01 • 분석의 목표

## 정상에서 벗어난 순간을 찾는 것

온도가 위험 수준을 넘은 시점, 진동이 비정상으로 커진 구간
모두 조건 필터링.

02 • 결합의 힘

## 필터링 + 통계로 "전체의 몇 %가 기준 초과"

필터링과 통계가 결합될 때 진짜 분석이 시작 — 현장 지식이 데이터로 드
남.

# 실습 4. 저회전 이상 시점 걸러내기

**목표**

기준 아래로 떨어진 회전수만 조건으로 선별

**단계**

① 회전수와 토크 묶음을 각각 준비

② 회전수가 2000 미만 시점마다 판단해 참거짓 묶음 만들어 출력하기

③ 회전수가 2000 미만이거나 전류가 25을 초과 시점을 참거짓 묶음으로 만들어 출력하기

④ 조건을 만족하는 시점이 각자 몇 개인지 출력하기

⑤ 회전수 2000 미만은 전체의 몇 퍼센트인지, 회전수가 2000 미만이거나 전류가 25 초과 시점은 전체의 몇 퍼센트인지 구하기

**예상 결과**

저회전 횟수와 비율, 저회전이면서 고전류 횟수와 비율

posco

# 실습 4. 저회전 이상 시점 걸러내기 정답

**조건을 묶을 때는 각 조건을 반드시 괄호로 감싼다**

CODE

```
import numpy as np

# 19~28번째 측정 (실측값)
rpm = np.array([4603, 4603, 58, 4603, 4603,
                1216, 4603, 4603, 1217, 4603])
tq  = np.array([2, 2, 21, 2, 3, 2, 2, 3, 2, 147])
print(rpm[rpm < 2000])                  # [  58 1216 1217]
print(rpm[(rpm < 2000) | (tq > 100)])   # [  58 1216 1217 4603]
```

# 실습 5. 조건에 걸린 시점의 개수와 비율 세기

**목표**

조건을 만족하는 시점이 몇 개이고 전체의 몇 할인지 계산

**단계**

① 토크 열 개 묶음을 준비

② 토크가 10을 넘는지 시점마다 판단해 참거짓 묶음 만들기

③ 참거짓 묶음을 모두 더해 개수 구하기

④ 같은 묶음의 평균을 내어 비율 구하기

⑤ 개수와 비율을 출력해 참이 1로 계산된다는 점을 확인

**예상 결과**

조건에 걸린 시점 2개, 비율 0.2 출력

posco

# 실습 5. 조건에 걸린 시점의 개수와 비율 세기 정답

참거짓을 더하면 개수, 평균을 내면 비율 — 합계·평균 함수는 PART 03에서 자세히

CODE

```
import numpy as np

# 1~10번째 측정 토크 (실측값)
tq = np.array([2, 2, 21, 2, 3, 2, 2, 3, 2, 147])
high = tq > 10
print(high.sum())            # 2
print(round(high.mean(), 2)) # 0.2
```

# 03

# 기초 통계

평균·표준편차 · axis 방향

# 통계가 왜 필요한가

**수천·수만 개 값을 일일이 못 보니, 몇 개의 대표 숫자로 요약하는 일**

01 · 요약

## 수많은 값을
## 몇 개의 숫자로 압축

평균이 얼마인지, 값들이 얼마나 흩어졌는지
데이터의 특징을 한눈에.

02 · 핵심

## 정상을 알아야
## 이상을 안다

평소 평균이 70도임을 알아야 95도가 나왔을 때 "이상하다"고 판단
통계가 이상 탐지의 토대.

# 주요 통계가 알려주는 것

**각 통계는 데이터의 다른 측면을 보여줌 — 일상에서 이미 쓰는 개념**

| 통계 | 알려주는 것 |
| --- | --- |
| **평균 · 중앙값** | 대표적인 중심값 — 평균은 전체를 더해 나눈 값, 중앙값은 줄 세운 한가운데 값 |
| **최대 · 최소** | 값의 **범위** — 어느 폭에 퍼져 있는지 |
| **표준편차** | **흩어진 정도** — 값들이 평균에서 얼마나 멀어지는지 |

중심 · 범위 · 흩어짐 세 측면 — NumPy는 이 모든 것을 함수 하나로 계산

posco

# 합계와 평균(mean)

**전체를 더하고 개수로 나눈 대표값**

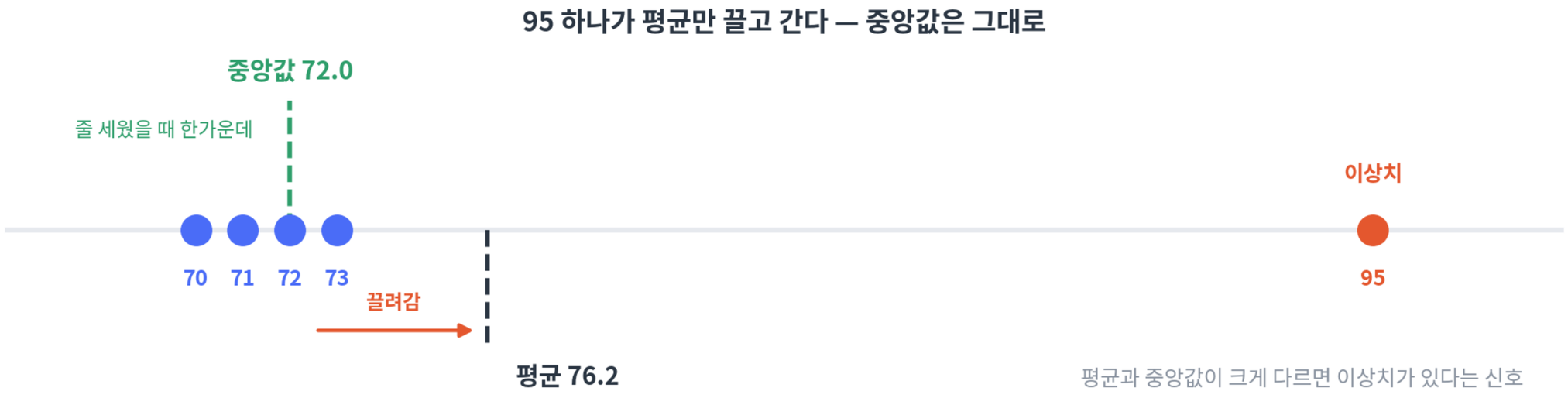
CODE

```
s = np.array([70, 72, 71, 95, 73])
print(s.sum())
# 출력: 381
print(s.mean())
# 출력: 76.2
```

95라는 높은 값 하나 때문에 평균이 76.2 — 소수점이 길면 round로 자리를 다듬어 읽음

# 중앙값(median)

**값을 줄 세웠을 때 한가운데 오는 값**




중앙값 72는 대부분 값에 가까움 — 이상치가 있어도 한가운데는 변하지 않음

# 중앙값(median)

값을 줄 세웠을 때 한가운데 오는 값 — 다른 통계와 달리 np.median 형태

CODE

```
s = np.array([70, 72, 71, 95, 73])
print(np.median(s))
# 출력: 72.0
print(s.mean())
# 출력: 76.2
```

중앙값 72는 대부분 값에 가까움 — 이상치가 있어도 한가운데는 변하지 않음

posco

# 중앙값의 강점

**두 통계를 함께 구해 비교하는 습관 —데이터 건강 점검**

01 • 강점

## 이상치에 흔들리지 않음

맨 끝 값이 95든 950이든 한가운데 위치는 그대로.
줄의 중앙은 극단값과 무관.

02 • 활용

## 평균과 중앙값이 크게 다르면 이상치가 있다는 신호

비슷하면 데이터가 고름
크게 다르면 한쪽으로 치우친 이상치 존재.

# 최대·최소·범위

**가장 큰 값·작은 값, 그 차이로 폭 파악**

CODE

```
s = np.array([70, 72, 71, 95, 73])
print(s.max(), s.min())
# 출력: 95 70
print(s.max() - s.min())
# 출력: 25
```

범위가 크면 넓게 퍼짐 — 단 하나의 극단값이라 평균·표준편차와 함께 봐야 정확한 그림

# 분산(var)

**값들이 평균에서 흩어진 정도를 하나의 숫자로**

CODE

```
stable = np.array([70, 71, 70, 72, 71])
unstable = np.array([60, 85, 65, 95, 70])
print(round(stable.var(), 1))
# 출력: 0.6
print(round(unstable.var(), 1))
# 출력: 170.0
```

안정적이면 분산이 작고 불안정하면 큼 —값을 제곱해 구하므로 단위가 달라지는 것이 한계

posco

# 표준편차(std)

**분산의 제곱근 — 원래 단위로 흩어진 정도 표현**

CODE

```
s = np.array([70, 72, 71, 95, 73])
print(round(s.var(), 2))
# 출력: 89.36
print(round(s.std(), 2))
# 출력: 9.45
```

"값들이 평균에서 약 9.45도 흩어져 있음" —원래 단위(도)로 바로 해석

# 표준편차와 이상 탐지

**실무에서 분산보다 표준편차를 훨씬 자주 사용 — 이상 탐지의 기초**

01 • 해석성

**분산을 원래 단위로 되돌린 값**

해석하기 쉬워 실무에서 표준편차를 훨씬 자주 사용.

02 • 이상 탐지

**평균에서 표준편차의 2~3배 벗어나면 이상치**

값이 고르게 퍼져 있을 때 쓰는 경험 규칙 — 평균에서 몇 배 떨어졌는지로 판단

평균 +표준편차 → "정상의 정의" —이 정의 위에 이상 탐지가 세워짐

# 평균과 표준편차, 함께 보기

**평균이 같아도 표준편차가 다르면 전혀 다른 데이터**

| 설비 | 평균 | 표준편차 |
| --- | --- | --- |
| 설비 A | 70도 | 1도 (안정) |
| 설비 B | 70도 | 15도 (불안정) |

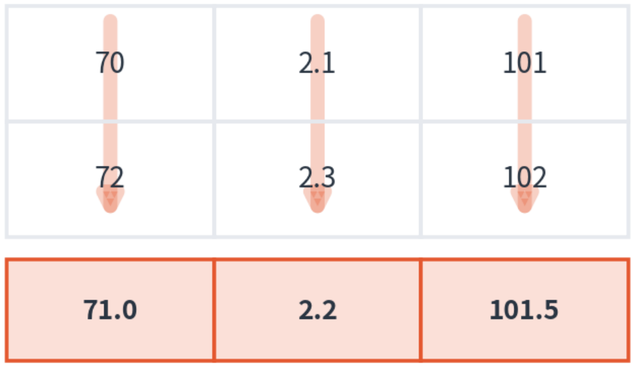
평균만 보면 똑같음 — 평균은 중심, 표준편차는 흩어짐. 둘 다 흔들리면 상태 변화 신호

# axis 개념 (행·열 방향)

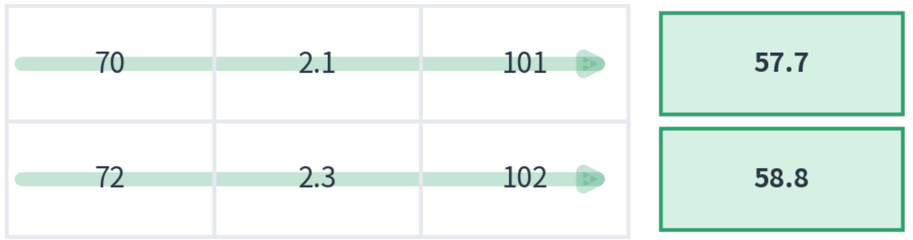
**통계를 어느 방향으로 낼지 정하기 — axis 생략 시 전체 평균**

**헷갈리면 결과 개수로 구분 — 2행 3열이면 axis=0은 3개, axis=1은 2개**

**axis=0 — 행 따라 아래로**




**axis=1 — 열 따라 옆으로**




행마다 값 하나씩 → 결과 2개 (시점별)

posco

# axis 개념 (행·열 방향)

**통계를 어느 방향으로 낼지 정하기 — axis 생략 시 전체 평균**

CODE

```
mat = np.array([[70, 2.1], [72, 2.3]])
print(mat.mean(axis=0))
# 출력: 열별(센서별) 평균
print(mat.mean(axis=1))
# 출력: 행별(시점별) 평균
```

axis=0 — 열별(센서별) · axis=1 — 행별(시점별)

# axis= 0과 axis= 1

**헷갈리면 결과 개수로 구분 — 센서별 통계가 가장 잦음 (axis=0)**

01 · 가장 자주 쓰는 방향

## axis=0 — 행 따라 아래로 열별(센서별) 결과

한 열 안의 모든 행 값을 모아 하나로 합침 — 열마다 값 하나씩 남음.

02 · 행 방향

## axis=1 — 열 따라 옆으로 행별(시점별) 결과

2행 3열 배열에서 axis=0은 값 3개, axis=1은 값 2개.

posco

# 기초 통계 함수 정리

**중심 •범위 •흩어짐 —세 측면을 함께 봐야 데이터 성격이 드러남**

| 통계 | 함수 | 측면 |
| --- | --- | --- |
| 평균 | `mean` | 중심 |
| 중앙값 | `np.median` | 중심 |
| 최대•최소 | `max · min` | 범위 |
| 표준편차 | `std` | 흩어짐 |

형태 —대부분 배열 뒤에 점, 중앙값만 np.median · 표 모양은 axis로 방향 지정

# 실습 6. 센서별 평균과 표준편차 구하기

**목표**

표 모양 데이터에서 센서마다 따로 통계 계산

**단계**

① 회전수와 토크를 표 모양으로 준비

② 열마다 세로로 값을 모아 평균을 구하도록 방향을 지정

③ 같은 방향으로 표준편차도 구하기

④ 두 결과를 소수점 첫째 자리까지 다듬어 출력

⑤ 결과에 값이 두 개씩 나오는 이유를 확인

**예상 결과**

회전수•토크 각각의 평균과 표준편차가 두 개씩 출력

posco

# 실습 6. 센서별 평균과 표준편차 구하기 정답

**방향을 열 쪽으로 잡으면 열마다 값 하나씩 남는다**

CODE

```
import numpy as np

# 4시점 × 2센서(회전수, 토크) — 실측값
data = np.array([[4603, 2], [4967, 21],
                 [3632, 147], [58, 5]])
print(np.round(data.mean(axis=0), 1))  # [3315.    43.8]
print(np.round(data.std(axis=0), 1))   # [1942.7   60. ]
```

# 파일에서 배열로 불러오기(np.loadtxt)

**저장된 데이터를 배열로 읽어 오기 — 구분 기호·건너뛸 줄·가져올 열 세 가지를 지정**

CODE

```
rpm = np.loadtxt('data/10_mct_tool.csv',
                 delimiter=',',   # 값을 나누는 기호
                 skiprows=1,      # 첫 줄(제목)은 건너뜀
                 usecols=4)       # 5번째 열 = 회전수
print(rpm.shape, rpm.dtype)
# 출력: (40,) float64
```

열 번호도 0부터 — 회전수는 다섯 번째 열이라 4 · 두 열을 함께 가져오려면 usecols=(4, 5)

# 실습 7. 파일에서 회전수를 불러와 통계 내기

**목표**

저장된 공정 데이터에서 한 열만 불러와 기초 통계 계산

**단계**

① 공정 데이터 파일에서 회전수 열 하나만 불러오기 (첫 줄 제목은 건너뛰고)

② 불러온 값들의 평균을 구하기

③ 값들이 평균에서 얼마나 흩어졌는지 구하기

④ 가장 작은 값과 가장 큰 값을 구하기

⑤ 네 숫자를 출력하고, 최솟값이 유난히 작은 이유를 생각해 보기

**예상 결과**

회전수 평균 4212.6, 표준편차 1144.9, 최소 58.0, 최대 4987.0 출력

posco

# 실습 7. 파일에서 회전수를 불러와 통계 내기 정답

**표준편차가 1144.9로 큰 것은 저회전 시점이 섞여 있다는 신호**

CODE

```
import numpy as np

# 출처: KAMP 제조AI 데이터셋 — MCT 가공 Tool 관리
rpm = np.loadtxt('data/10_mct_tool.csv',
                 delimiter=',', skiprows=1, usecols=4)
print(round(rpm.mean(), 1))   # 4212.6
print(round(rpm.std(), 1))    # 1144.9
print(rpm.min(), rpm.max())   # 58.0 4987.0
```

# 실습 8. 걸러낸 값들만 모아 통계 내기

**목표**

조건으로 값을 선별한 뒤 그 값들의 통계 계산

**단계**

① 공정 데이터 파일에서 토크 열 하나만 불러오기

② 토크가 10을 넘는 값만 골라내기

③ 골라낸 값이 실제로 무엇인지 출력해 확인

④ 골라낸 값들의 개수와 평균을 구해 출력

⑤ 전체 평균과 비교해 얼마나 차이 나는지 확인

**예상 결과**

골라낸 값 네 개, 개수 4, 평균 83.8 출력

posco

# 실습 8. 걸러낸 값들만 모아 통계 내기 정답

**전체 토크 평균은 10.7 — 걸러낸 뒤의 평균 83.8과 크게 다르다**

CODE

```
import numpy as np

# 출처: KAMP 제조AI 데이터셋 — MCT 가공 Tool 관리
tq = np.loadtxt('data/10_mct_tool.csv',
                delimiter=',', skiprows=1, usecols=5)
high = tq[tq > 10]
print(np.round(high, 1))       # [ 21. 147. 146.  21.]
print(high.size, round(high.mean(), 1))   # 4 83.8
```

# 실습 9. 불러오기부터 통계까지 한 흐름으로

**목표**

파일 읽기·구조 확인·필터링·통계를 하나로 이어 수행

**단계**

① 공정 데이터 파일에서 회전수와 토크 두 열을 함께 불러오기

② 불러온 표가 몇 줄 몇 칸이고 어떤 종류의 값인지 확인

③ 표에서 회전수 열만 따로 꺼내기

④ 회전수가 2000 아래로 떨어진 시점만 골라내기

⑤ 골라낸 시점의 개수와 평균 회전수를 전체 평균과 나란히 출력

**예상 결과**

구조 (40, 2) float64, 이상 시점 4개, 이상 평균 933.0, 전체 평균 4212.6 출력

posco

# 실습 9. 불러오기부터 통계까지 한 흐름으로 정답

**이상 4건이 전체 평균을 4212.6까지 끌어내린 셈 — 걸러내야 정상 수준이 보인다**

CODE

```
import numpy as np

# 출처: KAMP 제조AI 데이터셋 – MCT 가공 Tool 관리
data = np.loadtxt('data/10_mct_tool.csv',
                  delimiter=',', skiprows=1, usecols=(4, 5))
rpm = data[:, 0]
print(data.shape, data.dtype)      # (40, 2) float64
low = rpm[rpm < 2000]
print(low.size, round(low.mean(), 1))   # 4 933.0
print(round(rpm.mean(), 1))             # 4212.6
```

posco

# 데이터 분석의 전체 흐름

**불러오기 → 구조 확인 → 추출 → 필터링 → 통계 요약, 이 다섯 단계가 분석의 뼈대**

STEP 1
### 불러오기
파일에서 배열로

`np.loadtxt`

→

STEP 2
### 구조 확인
shape·dtype으로
모양 점검

`.shape · .dtype`

→

STEP 3
### 추출
인덱싱·슬라이싱으로
원하는 부분만

`data[:, k]`

→

STEP 4
### 필터링
조건으로 정상·이상 구분

`v[v > 기준]`

→

STEP 5
### 통계 요약
평균·표준편차로
데이터 성격 압축

`.mean() · .std()`

중심(평균·중앙값) → 폭(최대·최소) → 흩어짐(표준편차) 순으로 읽으면 데이터 성격이 드러남